

656 - PROJET D'ANALYSE ET DE CONCEPTION

Introduction

-

mathieu.louis@brucity.education

Rappel charte IA de l'ICC



<https://www.iccbxl.be/web/data/uploads/pdf/ia-charte-icc-pdf.pdf>

Pause au milieu du cours ou finir plus tôt ?



Question:

Faisons-nous une pause de 19:30 à 19:40 ou préférez-vous terminer un peu plus tôt le cours ?

Rappel – organisation du cours



Date	Cours	Lieu	Sujet
15/09/2025	Cours 1	Présentiel	Introduction + analyse des besoins
22/09/2025	Cours 2	Présentiel	Modélisation des données
29/09/2025	Cours 3	Présentiel	Modélisation des processus
06/10/2025	Cours 4	Présentiel	Architecture solution
13/10/2025	Cours 5	Présentiel	Architecture logicielle
20/10/2025	Classe inversée - Cours 6	Distanciel	Théorie : méthode Agile
27/10/2025	Congé - Pas de cours		
03/11/2025	Cours 7	Présentiel	Annnonce projet de l'année + UX UI et sécurité
10/11/2025	Cours 8	Présentiel	Gestion de projet (méthodologies) + Travail sur projet
17/11/2025	Cours 9	Présentiel	Gestion de projet (planification et documentation) + Travail sur projet
24/11/2025	Cours 10	Distanciel	extra + Travail sur projet
01/12/2025	Cours 11	Présentiel	extra + Travail sur projet
08/12/2025	Cours 12	Présentiel	extra + Travail sur projet
15/12/2025	Cours 13	Distanciel	Travail sur projet
22/12/2025	Congé - Pas de cours		
29/12/2025	Congé - Pas de cours		
05/01/2026	Cours 14	Présentiel	Travail sur projet
12/01/2026	Cours 15	Présentiel	Présentation intermédiaire projet d'année
19/01/2026	Cours 16	Présentiel	Travail sur projet
26/01/2026	Cours 17	Distanciel	Travail sur projet
02/02/2026	Cours 18	Présentiel	Travail sur projet
09/02/2026	Cours 19	Présentiel	Présentation finale projet d'année
16/02/2026	Classe inversée - Cours 20	Distanciel	Exercices d'entraînement pour l'examen
23/02/2026	Congé - Pas de cours		
02/03/2026	Cours 21	Présentiel	Récapitulatif + Session Q&A
09/03/2026	Cours 22	Présentiel	Examen (Travail 50%, examen écrit 50%)

Présence à 60% des cours obligatoire !

Modalités d'évaluation du cours



Projet : 50% de la note finale (45% travail de groupe, 5% réflexion personnelle)

Examen : 50% de la note finale

Modalités d'évaluation du cours



L'évaluation de votre **projet** se base sur 4 dimensions principales.

- **Cohérence (30% de la note)**
- **Précision (20% de la note)**
- **Intégration (30% de la note)**
- **Autonomie (20% de la note)**

Ce qui est évalué : La logique d'ensemble de votre solution

- Les différentes parties s'articulent-elles bien ?
- Y a-t-il des contradictions dans votre proposition ?
- La solution répond-elle vraiment au besoin client ?

Exemples de manque de cohérence :

- Proposer une app mobile pour des utilisateurs âgés peu à l'aise avec le numérique
- Architecture microservices pour une petite app avec 100 utilisateurs
- Interface complexe pour une utilisation d'urgence

Pour réussir :

- Relisez votre dossier en vérifiant que tout se tient
- Faites le lien entre besoins client et choix techniques
- Justifiez chaque décision importante
- Vérifiez votre document sur l'orthographe et la grammaire !
-  **Si la tournure de vos phrases est trop 'artificielle' => problème...**

Ce qui est évalué : La qualité et exactitude de votre analyse

- Utilisez-vous correctement la terminologie technique ?
- Vos diagrammes respectent-ils les notations standards ?
- Les informations sont-elles exactes et détaillées ?

Exemples de manque de précision :

- Diagramme UML avec erreurs de notation
- "*L'app sera sécurisée*" sans préciser comment
- Budget "*pas cher*" au lieu de montants précis

Pour réussir :

- Utilisez les termes techniques vus en cours
- Respectez les conventions (ex. UML)
- Donnez des chiffres précis (délais, coûts, performances)

Ce qui est évalué : Votre capacité à mobiliser tous les concepts du cours

- Avez-vous utilisé les méthodes d'analyse enseignées ?
- La gestion de projet est-elle présente ?
- Sécurité, ergonomie, architecture sont-elles traitées ?

Exemples de bonne intégration :

- Diagrammes cas d'utilisation + modèle de données + wireframes
- Planning Gantt + analyse des risques + rôles équipe
- Architecture technique + contraintes sécurité

Pour réussir :

- Utilisez plusieurs types de diagrammes complémentaires
- Montrez que vous maîtrisez toute la chaîne analyse → conception → *réalisation* *
- N'oubliez aucun aspect important du projet

Ce qui est évalué : Votre initiative et réflexion personnelle

- Proposez-vous des solutions créatives ?
- Allez-vous au-delà du minimum demandé ?
- Montrez-vous une réflexion critique ?

Exemples d'autonomie :

- Identifier des besoins non exprimés par le client
- Proposer des alternatives techniques avec comparaison
- Anticiper les évolutions futures du système
- Aller un peu plus loin que l'énoncé

Pour réussir :

- Ne vous contentez pas de reproduire les exemples du cours
- Faites des recherches complémentaires
- Proposez plusieurs solutions et argumentez vos choix
- Utiliser des outils pour faire rapidement une maquette

Des questions ?

Rappel du cours précédent



Rappel du cours précédent



Le cycle de vie de projets IT existe (au moins) de 6 étapes

1. **Analyse des besoins** : Comprendre ce que veut vraiment le client
2. **Conception** : Dessiner la solution technique
3. **Développement** : Programmer l'application
4. **Tests** : Vérifier que tout fonctionne
5. **Déploiement** : Mettre en service
6. **Maintenance** : Corriger et améliorer

Rappel du cours précédent



Dans la partie « **Analyse** » on retrouve:

1. Les parties prenantes

- Primaires
- Secondaires

2. Les techniques de recueil d'informations

- L'entretien individuel
- L'atelier de travail (workshop)
- L'observation sur le terrain
- Le questionnaire

3. La spécification des besoins

- Rédaction d'un cahier des charges (**pourquoi, quoi, pour qui, comment mesurer le succès, quand, combien**)
- Les exigences
 - **fonctionnelles** : Ce que doit FAIRE l'application
 - **non fonctionnelles** : Comment l'application doit SE COMPORTER
- Le diagrammes de cas d'utilisation (acteurs, action, relation)

Chapitre 2

–

Modélisation et formalisation

Partie I

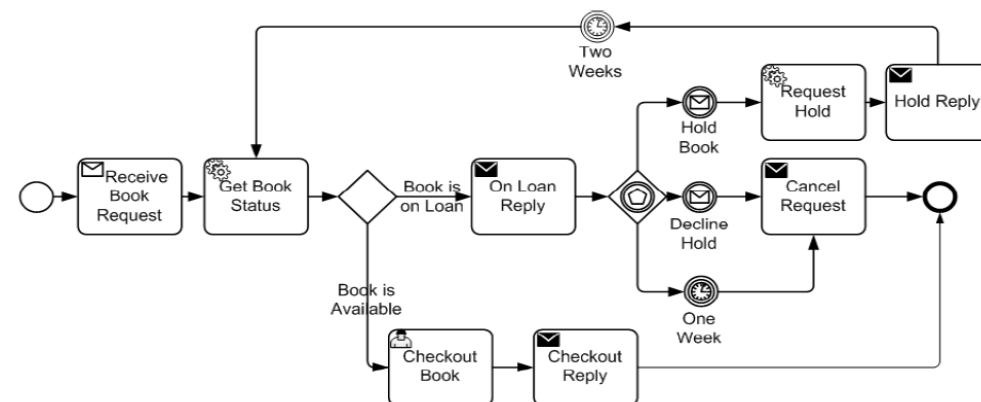
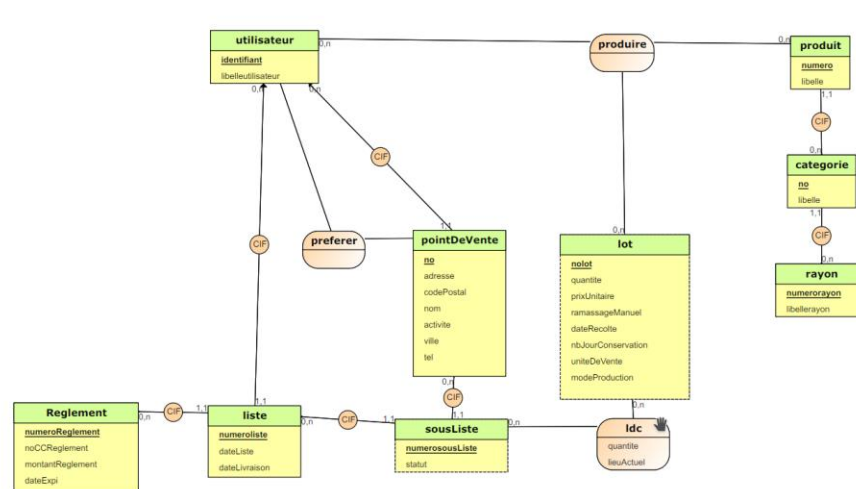


Description du chapitre

- 1 **Modélisation des données**
- 2 **Modélisation des processus**

Description du chapitre

- 1 Modélisation des données
- 2 Modélisation des processus



Où en sommes-nous ?

- Avant de développer une application, nous devons :
 - Comprendre quelles **informations (données)** nous manipulons
 - Comprendre quels **processus** nous devons automatiser

Modélisation des données



Qu'est-ce qu'une donnée ?



Une **donnée** est une information élémentaire que l'on doit stocker et manipuler.

Exemples concrets de données autour de nous :

- Dans une bibliothèque : livres, auteurs, emprunts, clients
- Dans un restaurant : plats, recettes, ingrédients, allergènes
- Dans une école : étudiants, notes, matières

Les entités et leurs attributs

Une **entité** représente un "objet" du monde réel que nous devons gérer.

Un **attribut** est une caractéristique de cette entité.

Exemple pratique - Système de gestion d'une pharmacie :

Entité : MEDICAMENT

1. *attribut 1*
2. *attribut 2*
3. *attribut 3*
4. *attribut 4*
5. *attribut 5*
6. *attribut 6*

Entité : CLIENT

1. *attribut 1*
2. *attribut 2*
3. *attribut 3*
4. *attribut 4*
5. *attribut 5*

Les entités et leurs attributs

Une **entité** représente un "objet" du monde réel que nous devons gérer.

Un **attribut** est une caractéristique de cette entité.

Exemple pratique - Système de gestion d'une pharmacie :

Entité : MEDICAMENT

1. Nom commercial (ex: "Dafalgan")
2. Principe actif (ex: "Paracétamol")
3. Dosage (ex: "500mg")
4. Prix (ex: 4.50€)
5. Stock disponible (ex: 25 boîtes)
6. Date d'expiration

Entité : CLIENT

1. Nom et prénom
2. Numéro de téléphone
3. Adresse
4. Date de naissance
5. Numéro de mutuelle

Les relations entre entités



Les entités sont liées entre elles par des **relations**.

Types de relations :

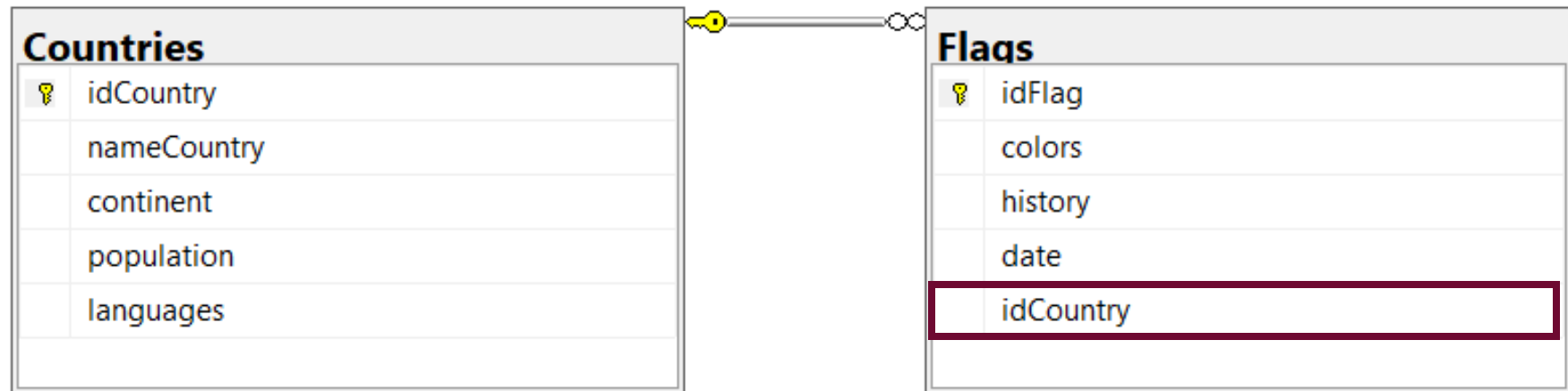
- Relation 1 vers 1 (un à un)
- Relation 1 vers plusieurs (un à plusieurs)
- Relation plusieurs vers plusieurs

Relation 1 vers 1 (un à un)

[1:1]

- Un passeport appartient à une seule personne
- Une personne n'a qu'un seul passeport

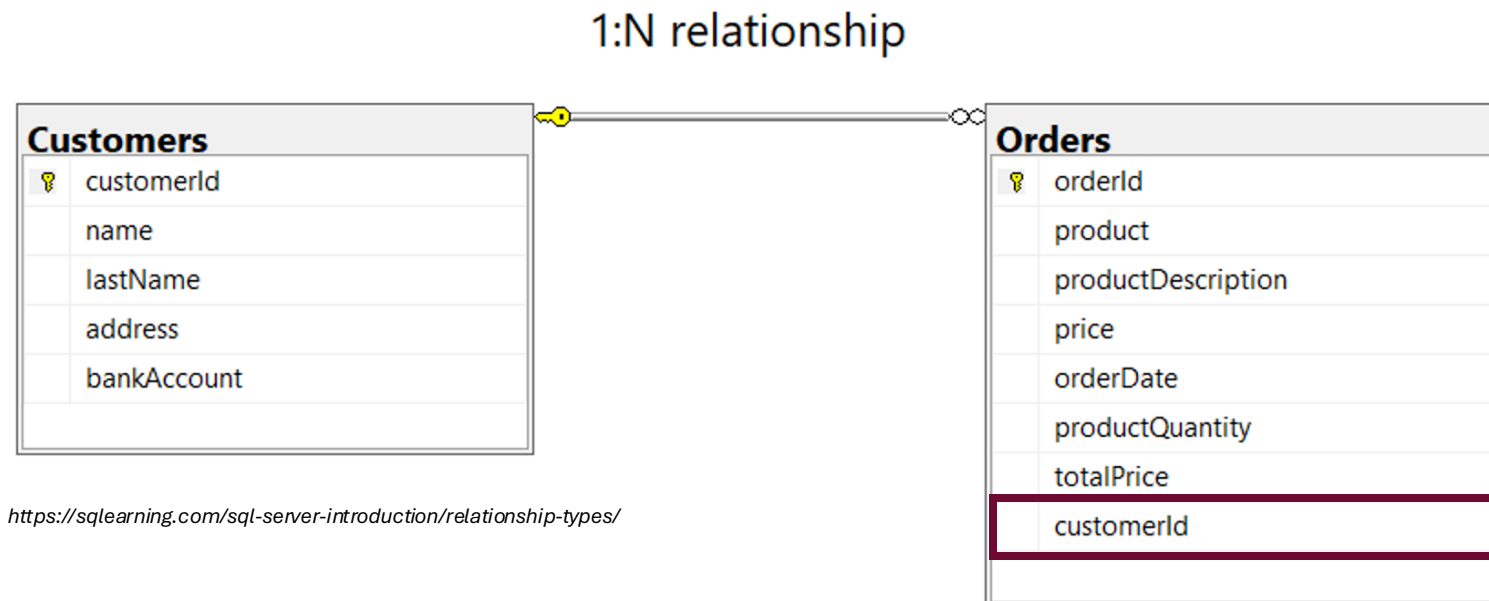
1:1 relationship



Relation 1 vers plusieurs (un à plusieurs)

[1:N]

- Un médecin peut avoir plusieurs patients
- Un patient n'a qu'un seul médecin traitant

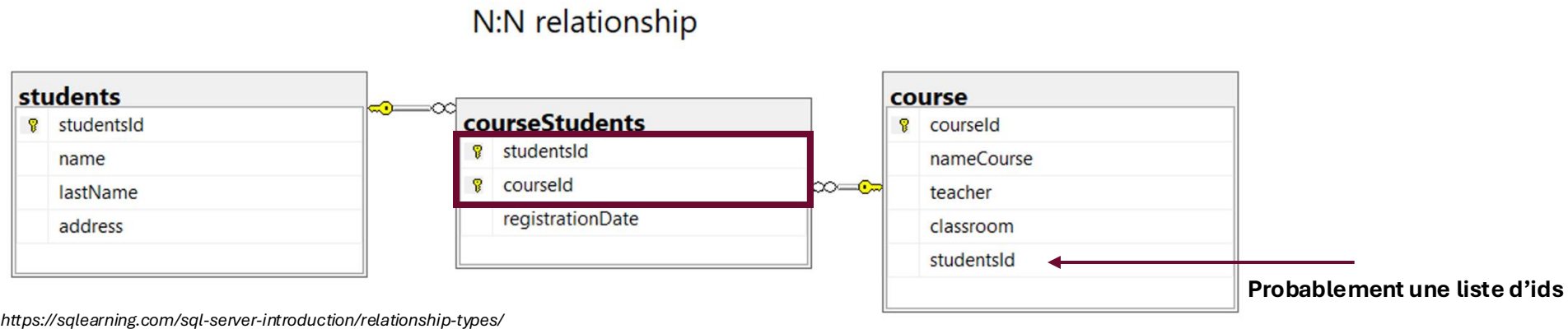


<https://sqllearning.com/sql-server-introduction/relationship-types/>

Relation plusieurs vers plusieurs

[N:N]

- Un étudiant suit plusieurs cours
- Un cours est suivi par plusieurs étudiants



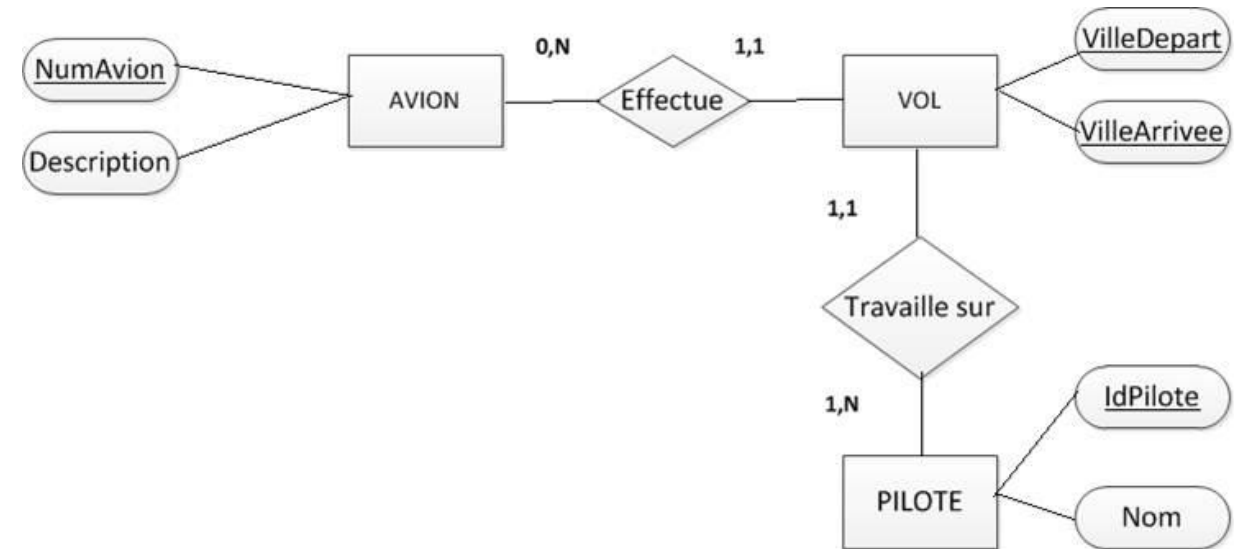
Le modèle entité-relation (MER)

Un diagramme qui représente visuellement :

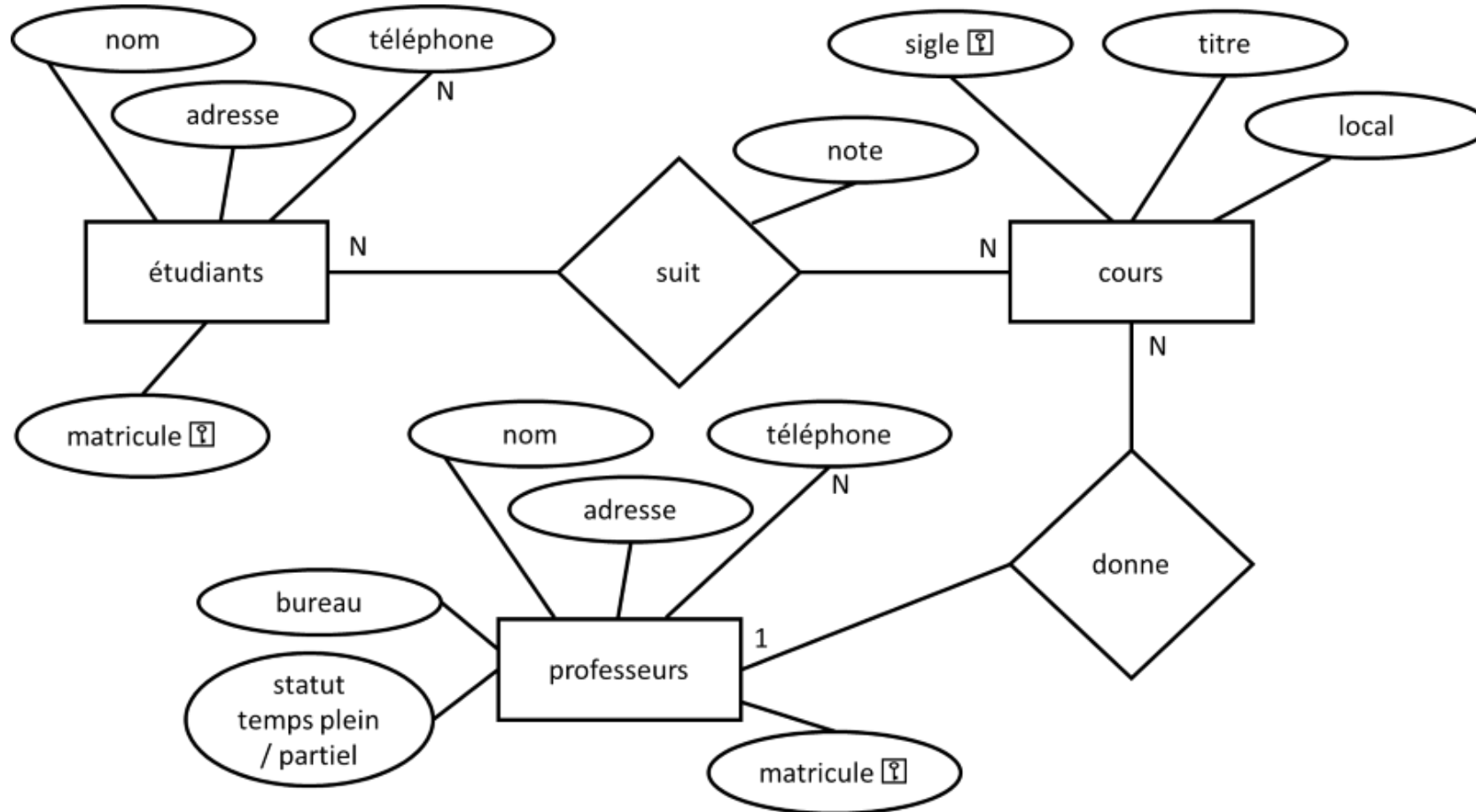
- Les entités (rectangles)
- Les attributs (ovales)
- Les relations (losanges)

Symboles de base :

- Rectangle = Entité
- Ovale = Attribut
- Losange = Relation
- *Trait simple = Relation optionnelle*
- *Double trait = Relation obligatoire*



Le modèle entité-relation (MER)



Des questions ?

Exercices pratiques



- **Exercice 8** : Créez le modèle entité-association pour un système de gestion d'un cabinet médical.
- **Exercice 9** : Pour un supermarché, identifiez 6 entités principales et leurs relations.

Du modèle conceptuel au modèle logique



Le problème : Le MER est excellent pour comprendre, mais ce n'est pas le meilleur outil pour guider les développeurs...

[C'est comme traduire une recette de cuisine en instructions pour un robot. Le robot a besoin d'instructions très précises, pas de "une pincée de sel".]

Transformer les entités et les attributs



Les règles de transformation

- Règle 1 : Chaque entité devient une table
- Règle 2 : Chaque attribut devient une rangée
 - Chaque rangée a un **type de données** (texte, nombre, date...)
- Règle 3 : Ajouter une clé primaire
 - **Clé primaire** = identifiant unique pour chaque ligne
 - Souvent un numéro automatique : id_medicament, id_client..

- **Relation 1 vers plusieurs** : On ajoute une **clé étrangère** dans la table "plusieurs".
 - **Exemple** : Un médecin a plusieurs patients
- **Relation plusieurs vers plusieurs** : On crée une **table de liaison**.
 - **Exemple** : Étudiants et cours

Exercices pratiques

- **Exercice 10** : Transformez-le modèle entité-relation du premier exercice en modèle logique (tables avec clés et relations)
- **Exercice 11 : Identification des entités et attributs**

Contexte : Vous devez concevoir un système pour une petite librairie.

Consigne : Identifiez les principales entités et leurs attributs pour gérer :

- Les livres en stock
- Les clients
- Les ventes

Il faut la liste des entités avec leurs attributs + justification vos choix

Diagrammes de classes (UML)



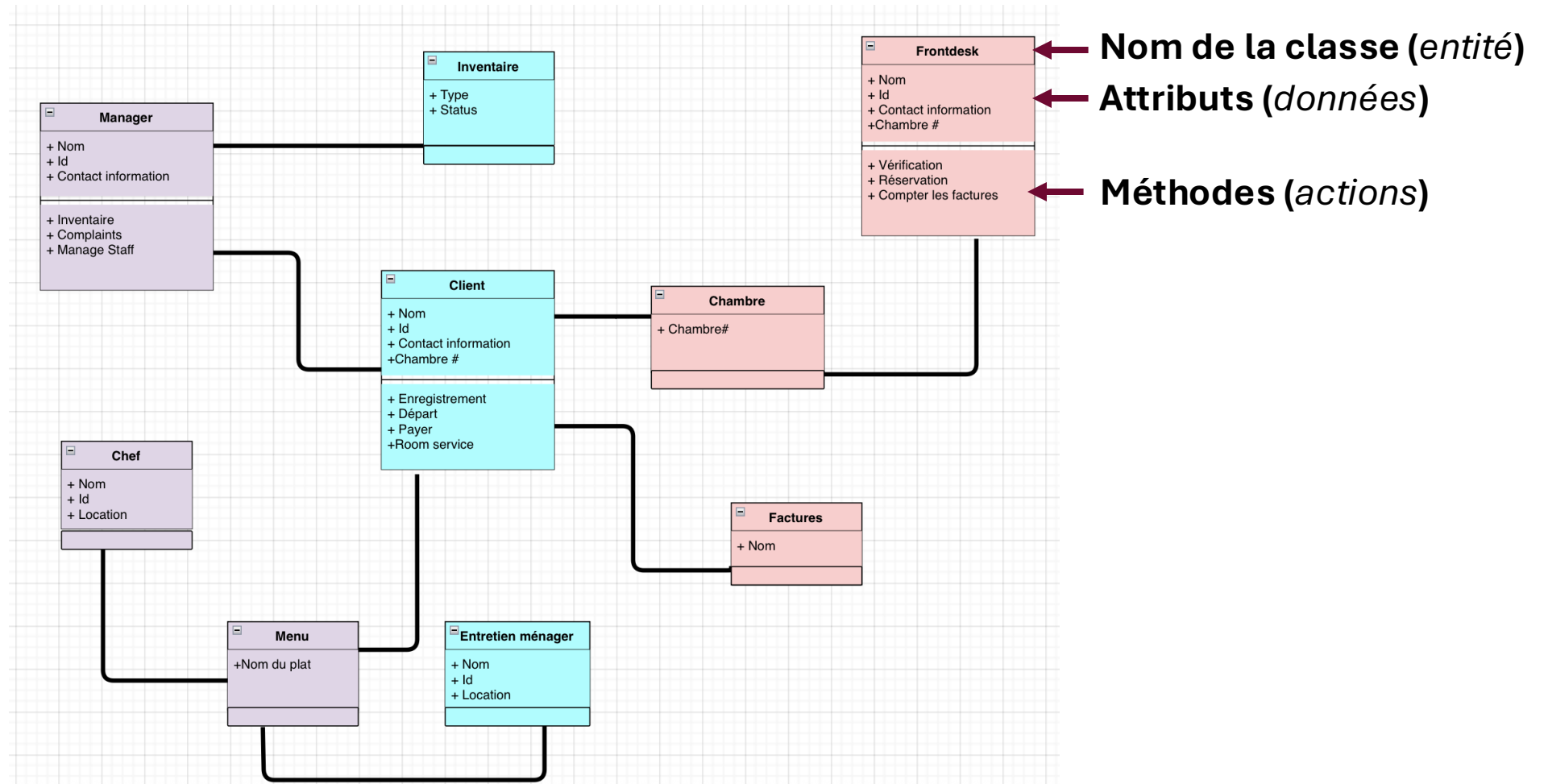
UML (Unified Modeling Language) est comme un langage universel pour dessiner des systèmes informatiques.

Pourquoi l'UML ?

- **Standardisé** : tout le monde comprend les mêmes symboles
- **Complet** : couvre tous les aspects d'un système
- **Précis** : permet de générer automatiquement du code

C'est assez similaire au MER mais c'est plus moderne et plus précis

Diagrammes de classes (UML)



Cardinalités UML

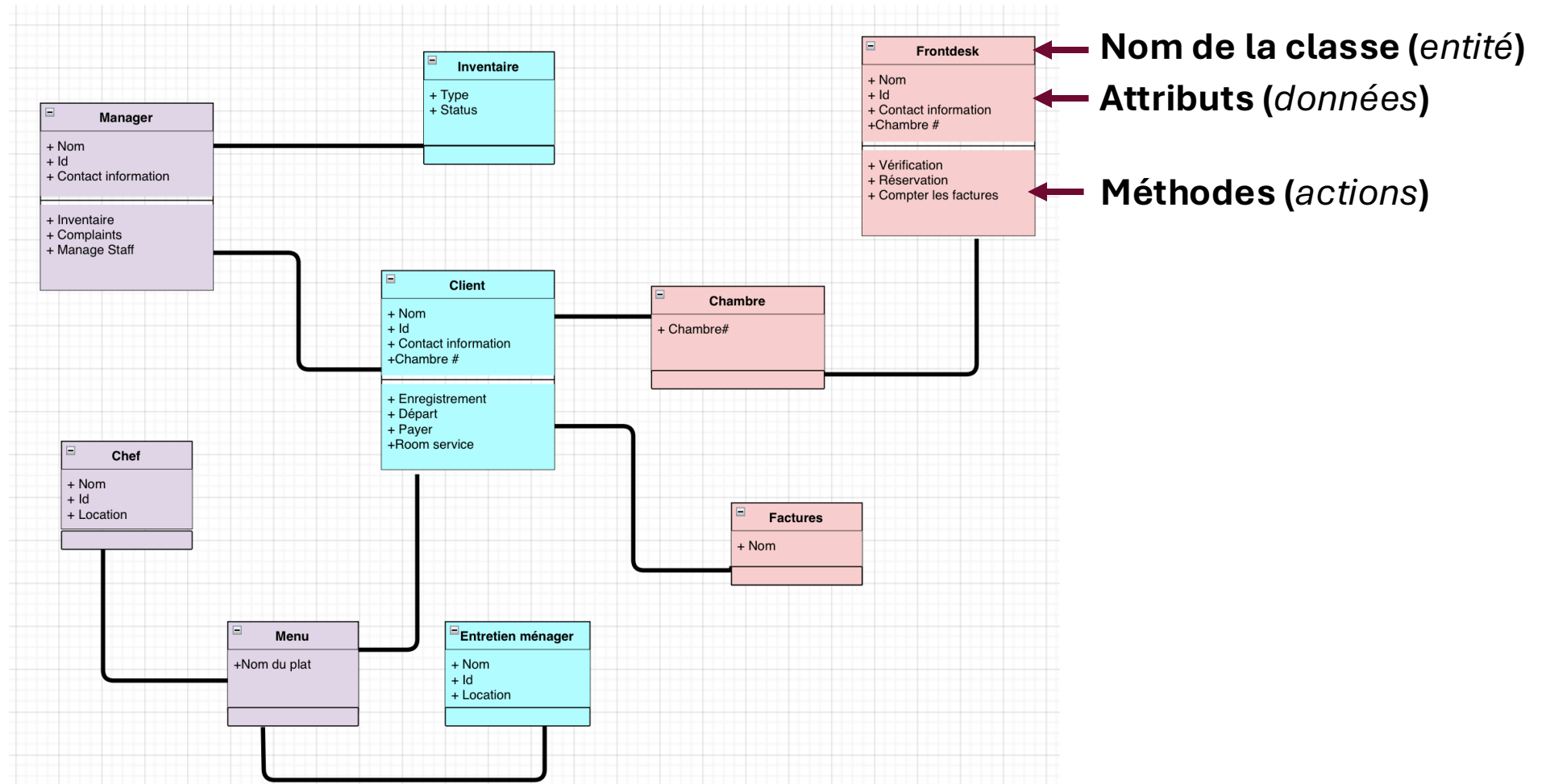


Notation : nombre minimum..nombre maximum

Exemples courants :

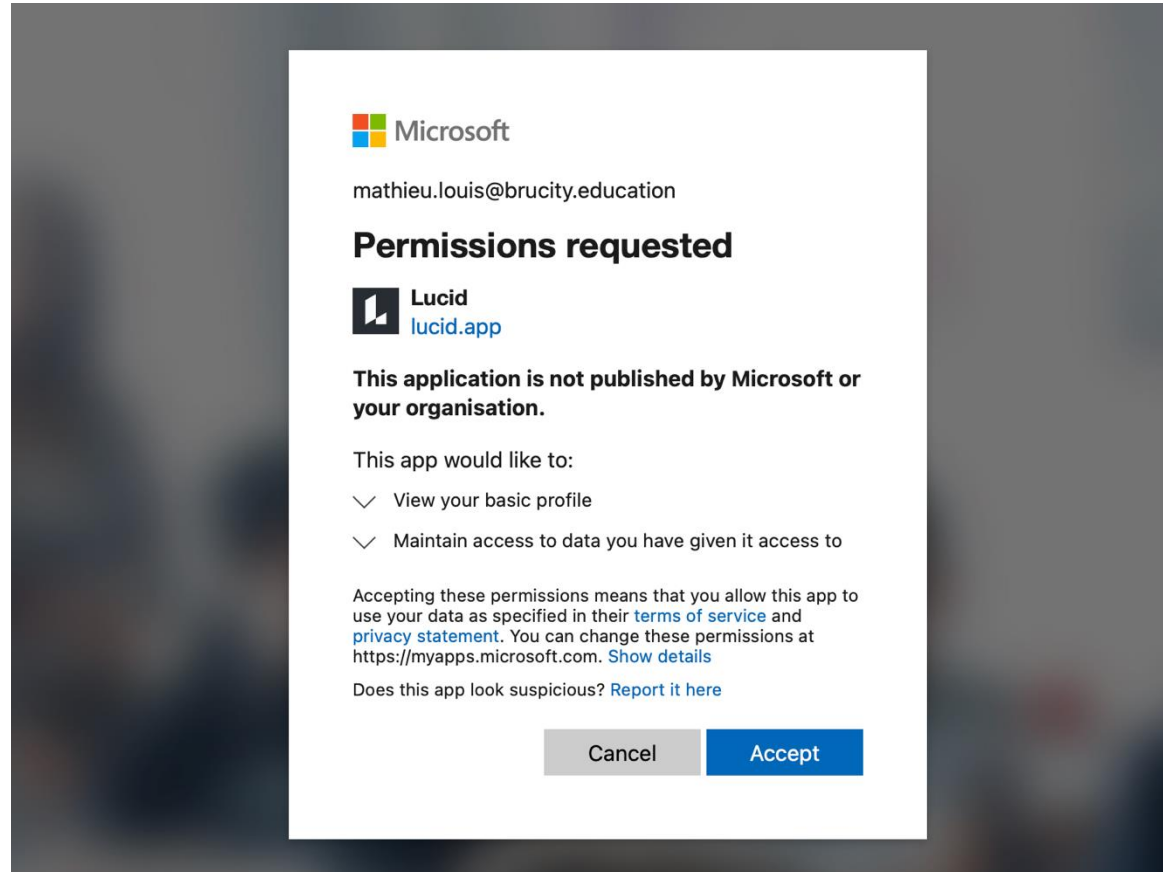
- 1 = exactement 1
- 0..1 = 0 ou 1 (optionnel)
- 1..* = au moins 1
- * ou 0..* = zéro ou plusieurs

Diagrammes de classes (UML)



Des questions ?

Travailler avec LucidChart



Travailler avec LucidChart



Sign up for free or log in with
your work email

Sign up free

Log in

Work email
mathieu.louis@brucity.education

Full name
anonyme



Password



Use 8 or more characters

Create account

or

 Sign up with Google

 Sign up with Microsoft

 Sign up with Slack

Ou avec app.diagram.net (anciennement draw.io)



<https://app.diagrams.net>

Exercices pratiques

Exercice 12 : Du MER -> modèle logique -> UML

Contexte : Vous avez identifié ces entités pour une librairie :

- LIVRE (titre, auteur, isbn, prix, stock)
- CLIENT (nom, prénom, email, téléphone)
- COMMANDE (date, statut)

Relations :

- Un client peut passer plusieurs commandes
- Une commande peut contenir plusieurs livres
- Un livre peut être dans plusieurs commandes

Consigne :

1. Dessinez le MER avec les bonnes cardinalités
2. Transformez-le en modèle logique (tables avec clés)
3. Créez le diagramme de classes UML correspondant